

Quantifying uncertainty in machine learning on nuclear binding energy

Mengyao Huang,^{1,*} Kyle A. Wendt,¹ Nicolas F. Schunck,¹ and Erika M. Holmbeck¹

¹*Lawrence Livermore National Laboratory, P.O. Box 808, L-414, Livermore, California 94551, USA*

Techniques from artificial intelligence and machine learning are increasingly employed in nuclear theory, however, the uncertainties that arise from the complex parameter manifold encoded by the neural networks are often overlooked. Epistemic uncertainties arising from training the same network multiple times for an ensemble of initial weight sets offer a first insight into the confidence of machine learning predictions, but they often come with a high computational cost. Instead, we apply a single-model uncertainty quantification method called Δ -UQ that gives epistemic uncertainties with one-time training. We demonstrate our approach on a 2-feature model of nuclear binding energies per nucleon with proton and neutron number pairs as inputs. We show that Δ -UQ can produce reliable and self-consistent epistemic uncertainty estimates and can be used to assess the degree of confidence in predictions made with deep neural networks.

DOI: [10.1103/tdk4-c4tp](https://doi.org/10.1103/tdk4-c4tp)

I. INTRODUCTION

The nuclear binding energy is a fundamental property of atomic nuclei and is the key driver of the energy released in nuclear reactions. An accurate and precise description of nuclear binding energy across the entire chart of nuclides is thus an important ingredient in modeling nuclear reactions relevant for medical applications [1], nuclear energy [2] and astrophysical studies [3]. For instance, complex networks of nuclear reactions are involved in neutron star mergers [4, 5] and core-collapsed supernovae [6]. However, current nuclear mass tables [7, 8] are neither complete nor sufficiently accurate for these astrophysical applications [9]. High-precision experimental measurement data are available near the valley of nuclear stability; while efforts are being made to continue expanding the measured region towards the nuclear drip lines [10–12], the nuclear masses of only about 40% of all nuclei predicted to exist have been measured. Theoretical mass models are either based on semi-empirical macroscopic-microscopic approaches [13–15], phenomenological microscopic models [16, 17], or non-relativistic energy density functional theory [18–20]. Masses predicted by these approaches are consistent within the experimentally measured region, where they are calibrated, but predictions vary significantly in neutron-rich or superheavy nuclei.

Recently, machine learning has become a powerful tool to reproduce several nuclear properties, including nuclear masses [21–25], charge radii [26, 27], and nuclear reaction cross sections [28, 29]. Unlike traditional methods, which rely on physical models, machine learning methods use neural networks that employ linear and nonlinear layers to directly fit the available data. The existence of apparent patterns and strong trends in nuclear binding energies across the chart of isotopes provides a strong incentive to employ machine learning to learn the correlations responsible for these patterns. However, extreme caution must be taken when extending predictions far away from the training data points on the nuclear chart. Barring the guidance of a physical model, a reliable uncertainty

quantification method is critical in deciding when and to what extent one can trust the machine learning results.

The two main sources of uncertainties are *epistemic* uncertainties due to limited data and a lack of knowledge of the best model in the hypothesis space, and *aleatoric* uncertainties that are irreducible by increasing data and knowledge. Distinguishing between epistemic and aleatoric uncertainties is hard to do in methods such as Gaussian processes [21], Bayesian neural networks [30, 31], and probabilistic networks [22] that generate uncertainties from posterior distributions. For example, Ref. [22] runs the probabilistic network 50 times for the resulting distribution as an indication of reproducibility, but it is not clear how much of this total uncertainty is caused by epistemic uncertainties alone. By contrast, deterministic networks lend themselves more easily to separating and evaluating epistemic uncertainties, although this has rarely been done in nuclear physics. In Ref. [23], the authors run the network 500 times with different random split of training and test data to obtain an estimate of them.

Given a neural network with a fixed number of layers and nodes, training the network for a set of random initial conditions allows for exploring the hypothesis space. Since training involves minimizing a function—the loss function—of a potentially large number of variables—the weights of the network—small differences in the initial numerical values of the weights can lead to substantially different solutions. Epistemic uncertainties can thus be estimated by independently training multiple copies of the same neural network with different initial weights and analyzing the spread of the results. This is an example of the ensemble methods [32–35] which offer a straightforward approach to estimate epistemic uncertainties. However, the multiple independent runs needed induce a high computational cost.

Single-model methods [36, 37] have been emerging as a more efficient way to estimate epistemic uncertainty. They only require running the network once with a unique set of initializations, and are able to generate both the predictions and epistemic uncertainties through that unique run. Δ -UQ [38] is one of these methods. By combining inputs with different constant biases, Δ -UQ generates uncertainties resembling those from ensemble methods but reduces total computational resource consumption by a factor of $\sim X$, where X is the size

* Contact author: huang59@llnl.gov

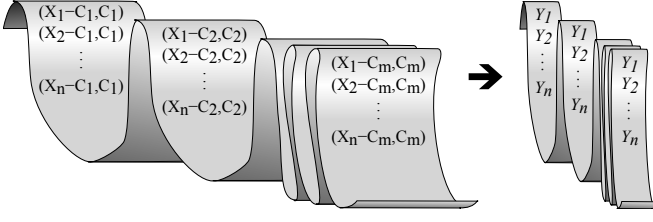


FIG. 1. Δ -UQ mapping: each initial input vector $\mathbf{X}_i$ is shifted by the anchor $\mathbf{C}_j$. The new input vector is formed by aggregating the shifted vector with the anchor, resulting in input dimensionality $2n_d$ and a dataset of size $n \times m$.

of the ensemble in the ensemble method. Δ -UQ also provides a convenient way to assess risk levels [39] to extend the extrapolation capabilities of usual single-model methods [36].

The goal of this work is to test the effectiveness of the Δ -UQ uncertainty quantification method in the case of deep neural network models of nuclear binding energy per nucleon (E/A). In particular, we show that Δ -UQ provides a quantitative indicator to assess the reliability of deep neural network predictions in extrapolations far from the training region.

The paper is organized as follows: in Sec. II, we briefly summarize the Δ -UQ method and describe how we generated the datasets used to train the neural network. Section III gives our results for two realistic scenarios. In the first scenario, the network is trained on a complete nuclear mass table from a theoretical calculation; in the second scenario, the network is trained on restricted data given by the Atomic Mass Evaluation [8] with the aim of quantifying uncertainties on predictions far away from it.

II. THEORY

In this section, we briefly introduce the Δ -UQ method, the construction of training, validation and testing sets, and the neural network architecture.

A. Summary of the Δ -UQ method

We use the Δ -UQ method to estimate the mean and uncertainty of the machine learning predictions [38]. The process is as follows. The input feature vectors $\{\mathbf{X}_i\}$ (in our case: $\mathbf{X}_i \propto (N_i, Z_i)$, see Sec. II D) are combined with a set of constant bias $\{\mathbf{C}_j\}$ called “anchors”, leading to multiple copies of input combinations $\{\mathbf{X}_i - \mathbf{C}_j, \mathbf{C}_j\}$, where $i = 1, \dots, n$ runs through all the input data points and j runs through all the anchors. For convenience, we call the expanded inputs Δ -UQ anchored inputs. The original input dimension n_d is thus doubled under the Δ -UQ scheme and the data length increases from n to $n \times m$, where n is the number of input data points and m is the number of anchors. Figure 1 illustrates schematically how Δ -UQ anchored inputs are generated.

After each epoch, the outputs contain results corresponding to different anchors. The loss is calculated using the mean

squared error (MSE) of the results from all the anchors,

$$\mathcal{L} = \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m (\hat{y}_{ij} - y_i)^2, \quad (1)$$

where $\hat{y}_{ij}$ is the model output for data i with anchor j , and y_i is the expected value of output i .

We employ the Adam optimization method [40], which adjusts the learning rate automatically in response to gradients calculated during the training. During training, the network parameters are updated to reduce the training loss. The training (validation) loss is calculated by applying (1) to the n_t (n_v) points of the training (validation) set and recorded after the network parameters are settled down after each epoch. The optimal network parameters are found when both losses reach the minimum and are stable within certain precision, where we allow for insignificant fluctuations around the true minimum due to the stochastic behavior of the Adam optimization algorithm.

After the optimal set of parameters of the neural network has been obtained, we compute the predictions. For each data point i , the Δ -UQ method generates a spread of m different values corresponding to the m anchors. One can calculate the mean value of these predictions μ_{Δ} for nucleus i ,

$$\mu_{\Delta i} = \frac{1}{m} \sum_j^m \hat{y}_{ij}, \quad (2)$$

and the standard deviation σ_{Δ} for nucleus i

$$\sigma_{\Delta i} = \sqrt{\frac{1}{m} \sum_j^m (\hat{y}_{ij}^2 - \mu_{\Delta i}^2)}, \quad (3)$$

where i runs from 1 to n for n being the total number of data points need to be evaluated.

B. Justification of Δ -UQ

In this section we give a simple demonstration of why the Δ -UQ method can estimate epistemic uncertainties. To simplify our analysis, let us consider only the first linear layer of the neural network because this layer is in direct contact with the Δ -UQ input transformation. For any given node in this first layer, the original result y is expressed as a function of the input feature x according to $y = kx + b$. With Δ -UQ the result becomes

$$y = (k, \mathcal{E}) \begin{pmatrix} C \\ x - C \end{pmatrix} + b = \left[\mathcal{E} + (k - \mathcal{E}) \frac{C}{x} \right] x + b, \quad (4)$$

where C is the Δ -UQ constant bias, k and $\mathcal{E}$ are the weights and b is the bias of the linear transformation. Each of these quantities can be multi-dimensional according to the problem. For the sake of simplicity, we neglect b in the current discussion. Training a single node in the first layer of the neural network means finding the optimal number k so that

$$\{kx_1, kx_2, kx_3, \dots\} \rightarrow \{y_1, y_2, y_3, \dots\}, \quad (5)$$

where $\mathbf{X} = \{x_1, x_2, x_3, \dots\}$ are the training inputs and $\mathbf{Y} = \{y_1, y_2, y_3, \dots\}$ are the targets. In the Δ -UQ method, the con-

stant biases $\mathbf{C}$ are selected from the training data, that is, $\mathbf{C} = \{x_1, x_2, x_3, \dots\}$. We can form the matrix $\mathbf{K}$ as

$$\mathbf{K} = \begin{bmatrix} \ell + (k - \ell) \frac{x_1}{x_1} & \ell + (k - \ell) \frac{x_2}{x_1} & \dots & \ell + (k - \ell) \frac{x_n}{x_1} \\ \ell + (k - \ell) \frac{x_1}{x_2} & \ell + (k - \ell) \frac{x_2}{x_2} & \dots & \ell + (k - \ell) \frac{x_n}{x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \ell + (k - \ell) \frac{x_1}{x_n} & \ell + (k - \ell) \frac{x_2}{x_n} & \dots & \ell + (k - \ell) \frac{x_n}{x_n} \end{bmatrix}, \quad (6)$$

and training involves finding the mapping

$$\mathbf{K}\mathbf{X} \rightarrow \mathbf{Y}. \quad (7)$$

This is still a mapping from $\mathbf{X}$ to $\mathbf{Y}$. The only change is that the initial weight k (= a single number) becomes a matrix element K_{ij} , where $i, j = 1, 2, \dots, n$ and n is the number of training data points. Each K_{ij} explores a different point in the multivariate surface in the hypothesis space at the vicinity of the original solution. It can be seen immediately that one solution is when n^2 weights reach an optimum with $k = \ell$. Then $\mathbf{K}$ reduces to n identical mappings characterized by k and the neural network has no epistemic uncertainty.

The diagonal terms in (6) can be simplified,

$$\mathbf{K} = \begin{bmatrix} k & \ell + (k - \ell) \frac{x_2}{x_1} & \dots & \ell + (k - \ell) \frac{x_n}{x_1} \\ \ell + (k - \ell) \frac{x_1}{x_2} & k & \dots & \ell + (k - \ell) \frac{x_n}{x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \ell + (k - \ell) \frac{x_1}{x_n} & \ell + (k - \ell) \frac{x_2}{x_n} & \dots & k \end{bmatrix}. \quad (8)$$

Therefore, if we just select the diagonal terms $K_{ii} = k$, we can go back to $k\mathbf{X} \rightarrow \mathbf{Y}$. Therefore, $k\mathbf{X} \rightarrow \mathbf{Y}$ is a subset of solutions of $\mathbf{K}\mathbf{X} \rightarrow \mathbf{Y}$. Thus we prove that Δ -UQ contains the original solution and it can explore the hypothesis space due to varying weights.

In practice, we can also recover the original solution of $k\mathbf{X} \rightarrow \mathbf{Y}$ by setting $\mathbf{C} = \mathbf{x}$ for every data point. Then Eq. (4) becomes

$$y = (k, \ell) \begin{pmatrix} x \\ 0 \end{pmatrix} + b = kx + b. \quad (9)$$

Since ℓ is irrelevant, the dimension of the first layer can be reduced by half so that the original neural network structure is recovered.

C. Δ -UQ and the ensemble method

Δ -UQ generates epistemic uncertainties that are comparable with the traditional ensemble method; the rigorous proof can be found in the Appendix of Ref. [38]. Here we show a simplified analysis to better understand the connections between the two methods.

Let us denote $\text{std}(\cdot)$ as the combination of multiplication of the subsequent layers and computation of the uncertainty. In Δ -UQ, the estimated uncertainty at the data point x_1 is

$$\Delta y_1 = \text{std}(kx_1, kx_1 + (k - \ell)(x_2 - x_1), \dots, kx_1 + (k - \ell)(x_i - x_1), \dots) \quad (10)$$

that is,

$$\Delta y_1 = \text{std}(kx_1, kx_1 + \Delta k(x_2 - x_1), \dots, kx_1 + \Delta k(x_i - x_1), \dots), \quad (11)$$

respectively. In other words, Δ -UQ is equivalent to creating an ensemble of n realizations of the same neural network, where the scaling factor k of the first layer of each realization i is obtained by adding a constant random deviation Δk scaled by the relative distance of the evaluated point from point i . In practice, all inputs and $\mathbf{C}$ form a full set of expanded inputs and they are fed to the neural network at each epoch in a single training process.

In the standard ensemble method, the uncertainty at point x_1 is simply $\Delta y_1 = \text{std}(kx_1, k_1x_1, \dots, k_ix_1, \dots)$, where k_i , etc., represent different initial weights. This can be recast into

$$\Delta y_1 = \text{std}(kx_1, kx_1 + \Delta k_1x_1, \dots, kx_1 + \Delta k_ix_1, \dots), \quad (12)$$

respectively, with $\Delta k_i = (k_i - k)$. Like Δ -UQ, the ensemble method creates a set of n realizations of the same neural network. However, in contrast to Δ -UQ, the scaling factor k of the first layer in each realization i is obtained by adding a different random deviation Δk_i . In the ensemble method, the other layers are also initialized randomly, but in Δ -UQ they are not affected by the input transformation.

Although Δ -UQ seems to have some limitation in exploring the hypothesis space, the variations produced after stochastic descent can be similar to the variations generated by the standard ensemble method, especially when the machine learning

model approaches a stable minimum in the loss function surface where the vicinity variations in all directions are similar. It turns out that if the machine learning with Δ -UQ run converges nicely for the training and validation sets, it is nearly impossible for the results from different C to deviate wildly, since the initial weights only differ by a scalar factor. On the contrary, the standard ensemble method trains each copy of the neural network individually, if different runs stop at the same number of epochs, the convergence behavior from each run might differ significantly. Therefore, the Δ -UQ method generally produces a smaller standard deviation compared to the ensemble method.

From a practical point of view, Δ -UQ doubles the input dimension. This change might affect the learning path as well as the convergence rate. Thus, it might not be fair to directly compare the uncertainty predicted by Δ -UQ and by the ensemble method. Based on that, we focus mainly on obtaining the self-consistency of mean and uncertainty for Δ -UQ, but not on getting the exact same mean and uncertainty as the standard ensemble method.

D. Datasets Construction

Our neural network is trained either on the AME2020 dataset [8] (AME) or on a synthetic dataset generated by density functional theory (DFT) calculations. Nuclei computed in DFT include all unstable nuclei predicted to exist between the proton and the neutron dripline. In this work, we will first use the DFT dataset to assess the validity of our method to estimate epistemic uncertainties with Δ -UQ. We will then simulate a realistic scenario of training on experimental data (as captured by the AME) and making predictions in unknown nuclei.

1. DFT calculations

In DFT, we considered the SLy4 parametrization of the energy functional [41]. Since the parametrization of this functional does not specify the pairing channel, we adopted a standard surface-volume, density-dependent pairing force and fitted the pairing strength of both neutrons and protons on the 3-point odd-even mass staggering formula; see [42] for additional details. We computed the binding energy of all nuclei with $2 \leq Z \leq 120$ between the proton and neutron dripline, where the neutron (proton) dripline is defined as the set of nuclei where the 2-neutron (proton) separation energy changes sign. The procedure to determine the ground state of all even-even nuclei is described in the Supplemental Material of [43]. The energy of odd or odd-odd nuclei is computed from blocking calculations in the equal filling approximation [44]. For both neutrons and protons, the five lowest quasiparticle excitations were considered, thus resulting in 25 different configurations for odd-odd nuclei. The ground-state energy for the odd or odd-odd nucleus is taken as the lowest among all blocking calculations. All calculations were performed with the HFBTHO solver [45]. Hereafter, this mass table is referred to as the DFT dataset.

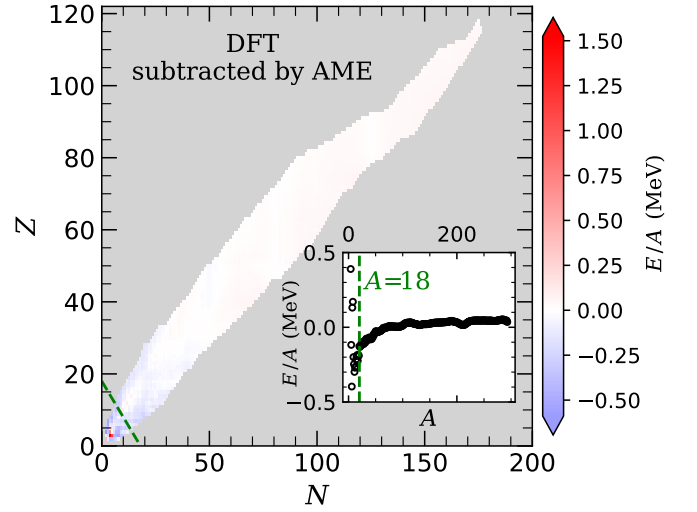


FIG. 2. The difference between DFT and AME E/A data. The inset shows the average difference calculated by averaging across each isobar.

It is well known that DFT is suboptimal for extremely light nuclei where correlations can be quite large. This is illustrated in Fig. 2, where we show the difference between DFT data and AME data. The inset shows the average difference between DFT and AME data as a function of mass number A . Both the landscape plot and the inset show that the difference jumps wildly when A is less than 18. Since the quality of the data can affect the training results, we choose to exclude data for nuclei with $A < 18$ in the DFT dataset. With this choice, the absolute value of the difference between the DFT dataset and AME data is less than 0.4 MeV, and the absolute value of the average difference for given A between the DFT dataset and AME data is less than 0.2 MeV. For a better comparison, we also construct AME dataset with $A \geq 18$.

2. Training, validation and testing sets

The size of the AME and DFT datasets is 3099 and 10393, respectively. The AME dataset is randomly split into a training set (90%) and a validation set (10%), which contain $n_t = 2789$ and $n_v = 310$ data points, respectively. Similarly, the DFT dataset in *region I*—that coincide with the AME dataset—is split into the same training and validation sets as the AME dataset, as shown in Fig. 3. The DFT dataset in *region II* forms the testing set of the DFT data.

The input features are normalized with Z-score normalization (standardization) to ensure stable and faster optimization, i.e., the input features $\mathbf{X}_i$ are

$$\mathbf{X}_i \equiv (x_{1i}, x_{2i}) = \left(\frac{N_i - \mu_N^t}{\sigma_N^t}, \frac{Z_i - \mu_Z^t}{\sigma_Z^t} \right), \quad i = 1, \dots, n. \quad (13)$$

where μ_N^t (μ_Z^t) and σ_N^t (σ_Z^t) are the mean and standard deviation of N (Z) in the training set: n is the number of data points that need to be evaluated. $n = n_t$ in the training phase, $n = n_v$

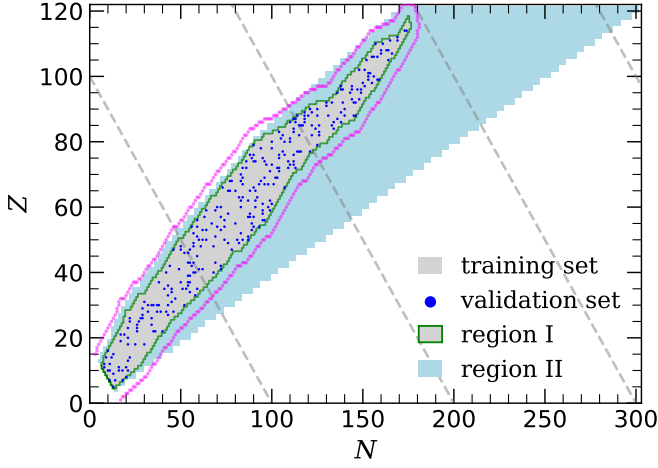


FIG. 3. Illustration of regions used for training, validation and testing. AME dataset is split into training set and validation set (blue dots) the same way as DFT dataset in region I. DFT dataset in region II forms the testing set of DFT data. The dashed gray lines indicate $A = 100, 200, 300, 400$ isobars, whose results will be presented later. The magenta dots indicate the farthest integer points within 5 nuclei away from the boundary nuclei of region I, with $A \geq 18$.

for validation after each epoch, and $n = n_{\text{tot}}$ for the evaluation of all data points after the training is complete.

For the training data, we subtract the mean of the binding energy per nucleon E/A from the actual value E/A to reduce the steepness of the sudden decrease of the loss in the first few epochs during the training. The actual output data is thus

$$Y_i = \left(\frac{E}{A}\right)_i - \mu_{E/A}^t, \quad i = 1, \dots, n, \quad (14)$$

where the mean value of the AME (DFT) training set $\mu_{E/A}^t = -8.056$ MeV (-8.045 MeV). After training, $(E/A)_i$ is recovered by adding $\mu_{E/A}^t$ back.

E. Network architecture

The neural network contains four composite layers. Each of the first three composite layers is composed of a linear layer, a nonlinear activation layer made of Sigmoid Linear Unit (SiLU) function, and a batchnorm layer [46] to stabilize the variance during training. The last layer consists only of a linear layer.

We recall that without applying Δ -UQ, the input dimension is $n_d = 2$ (for neutron and proton number); with Δ -UQ, the input dimension is $2n_d = 4$ (for neutron and proton number and their anchors). The output dimension is $p = 1$ since only the binding energy per nucleon E/A is fit. Each hidden layer contains 32 nodes. Data are fed into the network using multiple mini-batches with 64 data points per mini-batch. The batchnorm layer normalizes each mini-batch based on their mean and standard deviation before going through the next layer. It regulates the convergence speed regardless of the absolute values of the features.

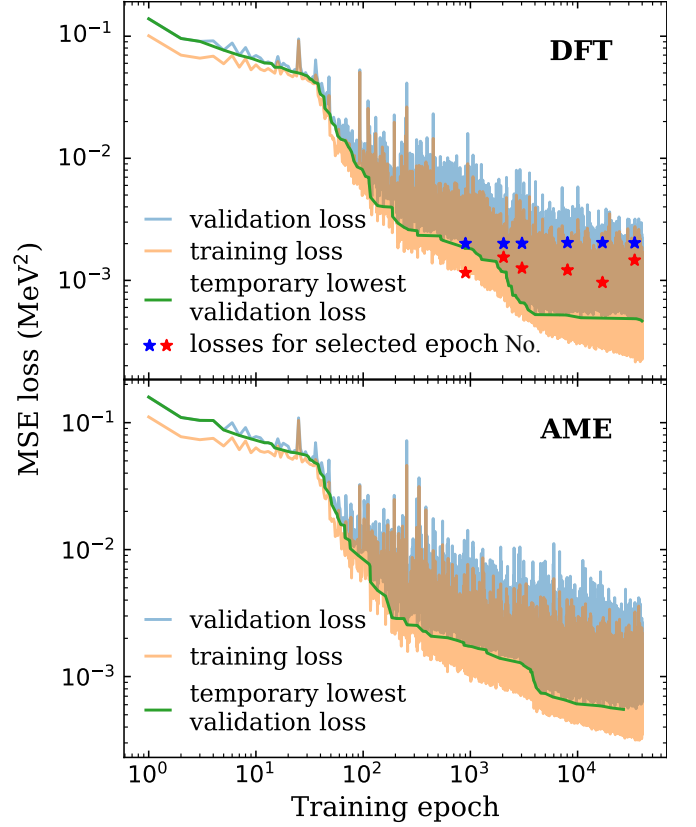


FIG. 4. Training on DFT or AME data: training and validation MSE loss as a function of the number of epochs. The blue (red) stars are the validation loss (training loss) values for selected epoch number with similar validation loss, which will be discussed later.

We choose initial learning rate to be 0.001 (default) and ϵ to be 8×10^{-6} . ϵ is a parameter of the Adam algorithm that effectively controls how fast the learning rate adapts to the changing gradient. We set ϵ to a higher value empirically instead of the default value 10^{-8} to prevent the learning rate from decaying too quickly. In this way, larger patterns of data can be learned before finer adjustments to decrease the loss, so that the learning process is more stable and beneficial to extrapolations.

III. RESULTS

We first discuss the convergence of the loss function during the training of the networks. Then we illustrate the machine learning results for both DFT and AME with Δ -UQ uncertainty quantification before comparing the Δ -UQ uncertainty quantification with the standard ensemble method.

A. The convergence of training process

Figure 4 shows the validation loss and training loss as a function of the number of training epochs when training either on the DFT (top panel) or AME data (bottom panel). The

TABLE I. Training loss, validation loss and their difference (MeV^2) for training on DFT data at selected epochs.

epoch	validation loss	training loss	difference
900	0.0020015	0.0011552	0.00084636
2039	0.0020024	0.0015453	0.00045709
3028	0.0020086	0.0012595	0.00074908
8045	0.0020343	0.0012146	0.00081966
17055	0.0020338	0.0009605	0.00107330
34009	0.0020264	0.0014621	0.00056429

similarities of both loss curves in the first 10^2 epochs indicate that the model is learning the same overall trend. After that, the loss curves become noisier and the model is learning the details of the data. We allow the process to run for 40000 epochs and save a copy of the neural network model parameters each time the validation loss becomes lower than at previous epochs. This corresponds to the curve marked “temporary lowest validation loss” in both panels. We stop the training process at 40000 epochs because this temporary lowest validation loss up to each epoch flattens out approaching 40000 epochs, as shown in Fig. 4.

When losses become noisy, the model is still continuously improving. We can see this by examining the results of a few epoch numbers during the training process. Here, we select epoch 900, 2039, 3028, 8045, 17055, 34009. They are chosen so that the difference in their validation losses is less than 10^{-4} MeV^2 , which is negligibly small compared to the loss fluctuations (Table I). However, the predictions are different, as illustrated in Fig. 5 in the particular case of the isobaric $A = 200$ nuclei.

It is important to emphasize that a smaller validation or training loss, or a smaller difference between validation and training loss, indicates convergence of the training but not nec-

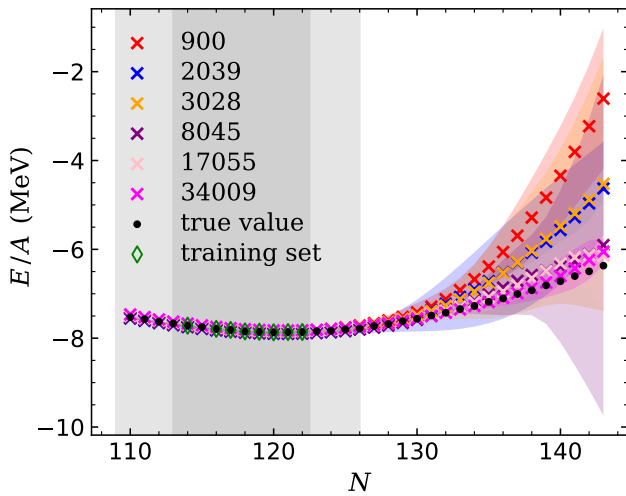


FIG. 5. Predictions of the binding energy per nucleon E/A for $A = 200$ isobars for the selected epochs listed in Table I. Crosses represent the mean value and the band indicates three standard deviation estimated with Δ -UQ.

essarily better results. For example, both the training and validation loss at epoch 900 are smaller than at epoch 2039, but epoch 2039 gives a better extrapolation. Similarly, the difference between validation and training loss at epoch 17055 is the largest in the set, while its predictions are closer to the true value in the extrapolation region. It can be observed that the model captures a new trend each time the temporary lowest validation loss has a significant drop, however, the oscillations above that temporary lowest validation loss will not change the quality of the fit, as long as the temporary lowest validation loss remains the same (until the next significant drop).

In the final plateau of temporary lowest validation loss, models produce very similar fits regardless of the local fluctuations of the loss. The best model is defined as the model that has the lowest validation loss. For DFT, it is obtained at epoch 39858 with training loss $2.46 \times 10^{-4} \text{ MeV}^2$ and validation loss $4.65 \times 10^{-4} \text{ MeV}^2$; for AME, the best model is obtained at epoch 26792 with training loss $3.55 \times 10^{-4} \text{ MeV}^2$ and validation loss $5.51 \times 10^{-4} \text{ MeV}^2$.

B. Training results with Δ -UQ uncertainties

Figures 6 and 7 show the prediction of the neural network for the binding energy per nucleon for the $A = 100$, $A = 200$, $A = 300$, and $A = 400$ isobars with Δ -UQ uncertainty quantification. These isobars correspond to the four diagonal dashed lines plotted in Fig. 3 and are representative of the results that we obtained. The training data represents a little more than half the entire set of the $A = 100$ isobars; conversely, it represents a little less than the entire set of the $A = 200$ isobars. None of the nuclei in the $A = 300$ isobars are in the training set, although some of them are not too far from it. Finally, the $A = 400$ isobars include nuclei that are very far from the training data. Figure 6 is obtained by training the network on the DFT dataset, while Fig. 7 is obtained by training on the smaller AME dataset.

Each panel in Figs. 6-7 shows the initial data (either DFT or AME) as open circles and the training data as losanges. The mean of the Δ -UQ prediction is shown as a plain line and computed from Eq.(2) where all $n_i = 2789$ anchor nuclei are included. For comparison, we also include the median prediction. We recall that the standard deviation of the Δ -UQ predictions is given by Eq.(11) or specifically, Eq.(3). For each panel, we choose the 3σ band to represent the Δ -UQ uncertainty. In Fig. 6, the true values of DFT in region II are covered by Δ -UQ uncertainty.

To better visualize the distribution of the results from all the anchors, the E/A range of each panel is discretized into 200 energy bins. The function $E/A(N)$ of each bin is represented as a colored line, the color of which is given by the number of counts in that bin (in log scale). The deviation of the distribution for each nucleus from a normal distribution is first shown by the difference between the mean and median. As we can see, the difference only shows up when the width of the uncertainty band significantly increases. The fact that all the mean values are within or on the margin of the middle 50% range (midspread, or interquartile range—IQR) further

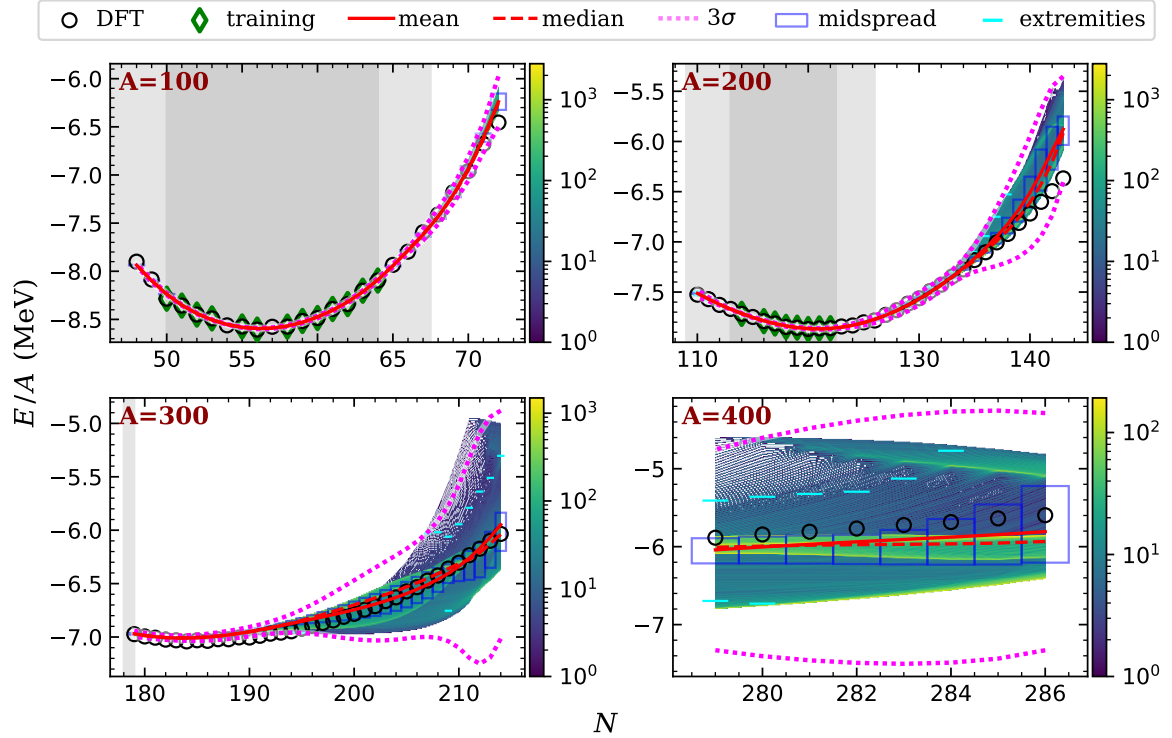


FIG. 6. E/A from training on DFT with Δ -UQ uncertainty quantification for selective isobars. See text for explanations.

indicates that the distributions are not very far from normal distributions. Another indicator of deviations from a symmetrical normal distribution is the *extremities* calculated by Tukey's 1.5 IQR rule [47]. In this rule, the maximum (minimum) is redefined as $Q_3 + 1.5 \times \text{IQR}$ ($Q_1 - 1.5 \times \text{IQR}$) when $Q_3 + 1.5 \times \text{IQR}$ ($Q_1 - 1.5 \times \text{IQR}$) is smaller (larger) than the true maximum (minimum) of the data, where Q_3 (Q_1) is the third (first) quartile of the data. And any data outside of the redefined maximum and minimum is a statistical outlier. These redefined extremities are shown in the figures. There seem to be more outliers above the maximum than below the minimum. However, these outliers have little effect in skewing the overall distribution because of their relatively lower density. The conclusion of this discussion is that the 3σ band originally derived under the assumption of normal distribution remains a reasonable quantity for uncertainty estimation in our case.

Figure 7 shows that training on AME data gives stable results close to the training data but that the uncertainty explodes very quickly at larger distance. Note that the range of the y-axis for the $A = 200$, $A = 300$ and $A = 400$ isobars is considerably larger in Fig. 7 than in Fig. 6. This can be partially attributed to the fact that the noises of DFT data have a consistent bias, as shown in Fig. 2, while the experimental data are more random. This randomness helps machine learning to overcome some local minima and may be the reason why predictions are better close to the training data. However, if we continue to extrapolate further, beyond a certain point, this randomness results in very different extrapolations for different anchors, so the uncertainty band explodes.

C. Comparison with the ensemble method

To compare the prediction of uncertainties of Δ -UQ with the standard technique of ensemble runs, we performed a set of 2789 runs using the DFT dataset—the exact same number of runs as anchors in the Δ -UQ method. In each run, the linear layers of the neural network are initialized with a different set of weights using Kaiming uniform [48], which is a scaled version of the uniform distribution popularly used to balance the variance between input and output, by applying a scale factor (Kaiming gain) to account for the variance change related to the input dimension of each linear layer in the deep neural network. The initial biases are also chosen from a uniform distribution scaled to match the magnitude of the output resulting from the Kaiming gain to ensure a stable and effective training process. The network layer-wise structure and dimensions are the same as the one we used in Δ -UQ method, except the input of the first linear layer is reduced back to n_d dimensional without requiring the input of anchors. After 40000 epochs, the best model is saved at the lowest validation loss. The results for the selected isobars are shown in Fig. 8 with the same conventions as Figs. 6-7. Without Δ -UQ, individual run takes approximately 2.7 GPU hours on an NVIDIA V100 GPU, and the ensemble of 2789 runs cumulatively consumes 7530 GPU hours. On the other hand, the single Δ -UQ run that encompasses 2789 anchors takes about 5 (2.4) GPU hours on an NVIDIA V100 (H100) GPU.

As expected, some of the ensemble runs produce exceptionally large deviations in the extrapolation region that are visible as additional colored lines in each panel. Even though the tip-

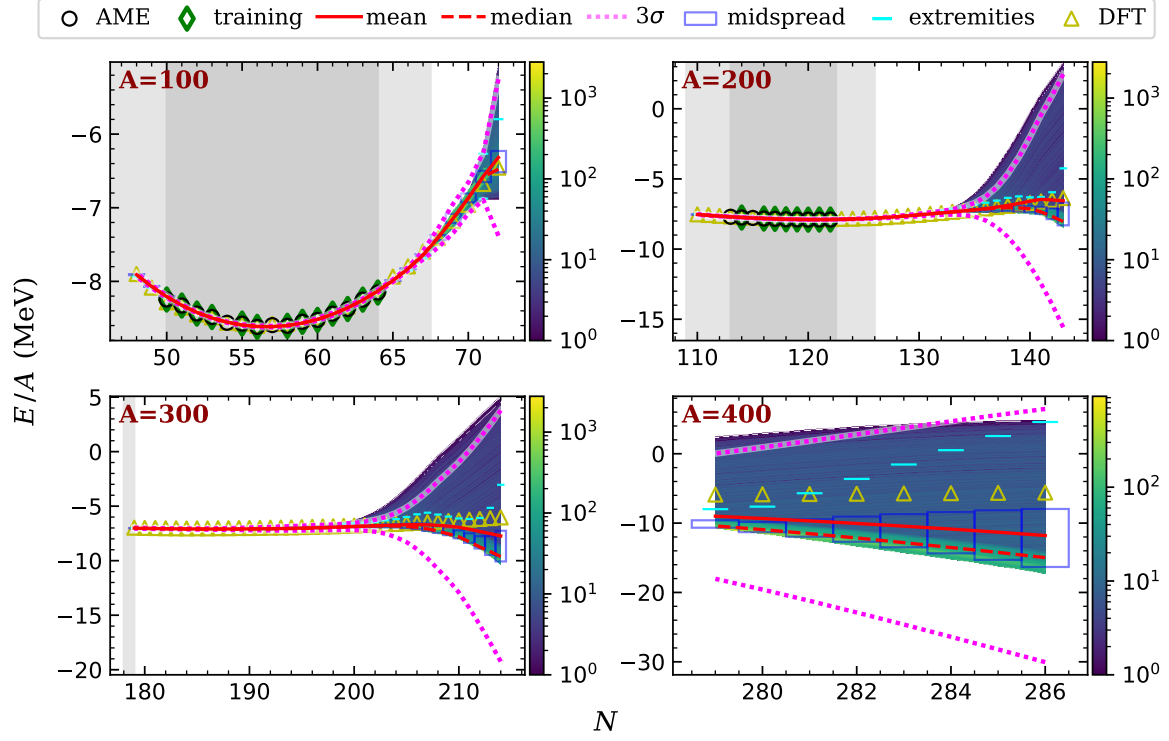


FIG. 7. Same as Fig. 6, only from the AME training dataset.

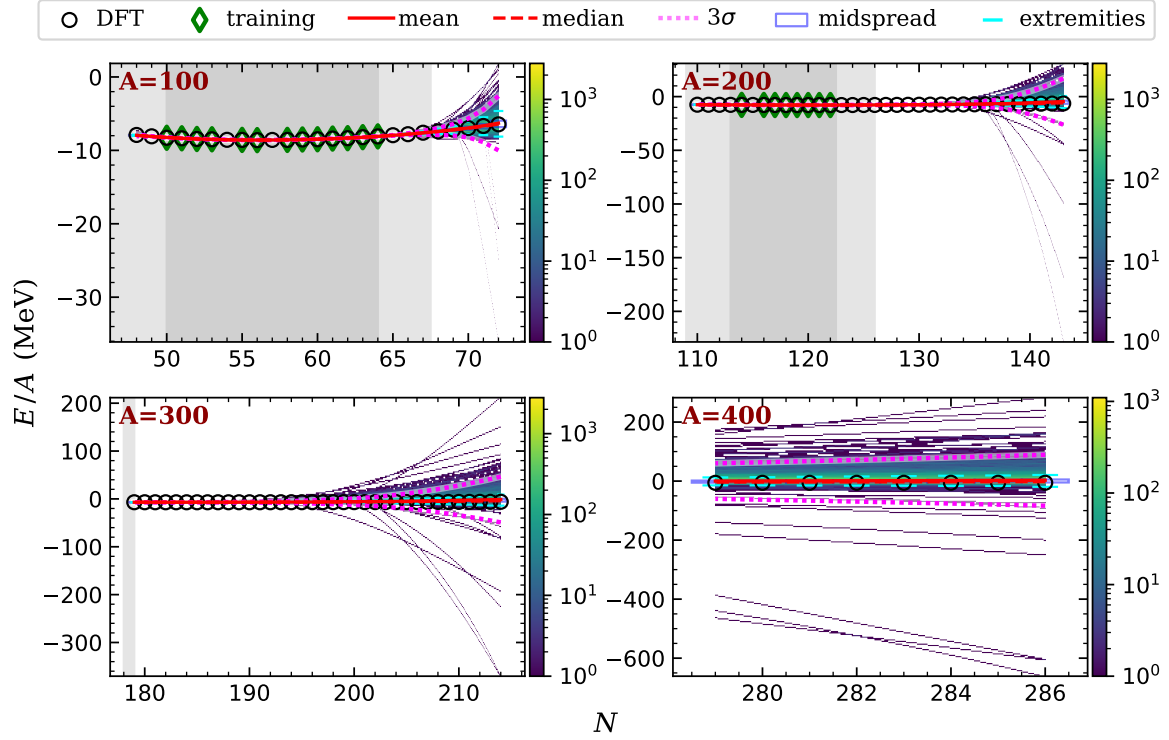


FIG. 8. E/A from training on DFT with ensemble uncertainty quantification for selective isobars. The color scale is used to visualize the distribution of the results from all the ensemble runs. See text for explanations.

ping point of the explosion in uncertainty, around $N = 134$ in $A = 200$ case for example, is the same in both Δ -UQ (Fig. 6) and ensemble method (Fig. 8), the increase in uncertainty in the ensemble method is considerably larger. The uncertainty band of the ensemble method can be one or two magnitudes larger than Δ -UQ. Note that each run, including the ones giving these very large deviations, still gives a good reproduction of the *training* data, as can be seen in Fig. 9, which shows the predictions for the $A = 200$ isobaric line only between $N = 110$ and $N = 126$. In other words, the ensemble method greatly overestimates the epistemic uncertainty of the network since it does not provide any quality control and does not account for the quality of the local minimum the network discovers during training. On the other hand, Δ -UQ method uncertainty represents the epistemic uncertainty of a good local minimum.

To get a better representation of this effect, we show in Fig. 10 the density distribution of the predictions for selected nuclei. For each nucleus, we plot the density distribution extracted from the Δ -UQ calculations and from the ensemble runs, with selected nuclei as examples. Five nuclei are chosen from $A = 200$ isobars, with $N = 114$ (^{200}Fl), $N = 115$ (^{200}At), $N = 126$ (^{200}W), $N = 130$ (^{200}Yb), and $N = 140$ (^{200}Nd). One additional nucleus from the $A = 400$ isobar, ^{400}Lv ($N=284$), is also included.

^{200}Fl and ^{200}At are included in the training and validation set, respectively, while ^{200}W is five nuclei away from the nearest training data in the testing set. In these three nuclei, Δ -UQ and the ensemble method predict comparable uncertainties, with the ensemble method uncertainty being slightly larger. By contrast, ^{200}Yb is eight nuclei away from the nearest training data along the same isobar, but the uncertainty is still relatively small, with the ensemble method uncertainty being around three times of the Δ -UQ uncertainty. Both ^{200}Nd and ^{400}Lv are far from any training data in the region where the epistemic uncertainty increases significantly. In these two nuclei, the ensemble method uncertainty is in fact orders of magnitude larger than what Δ -UQ predicts.

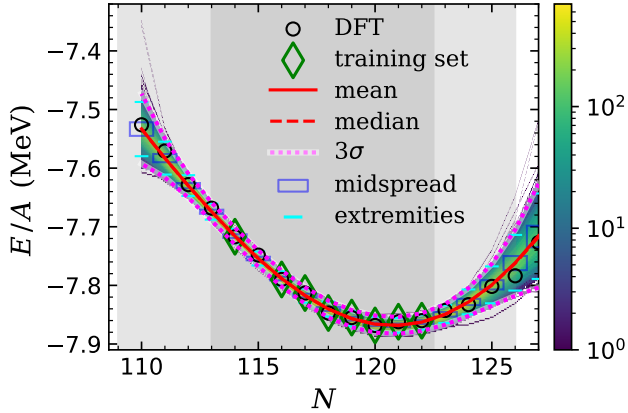


FIG. 9. A zoom-in figure of the ensemble method for $A = 200$ and $N < 127$, with the color-mapped bins adjusted for the E/A range in this window.

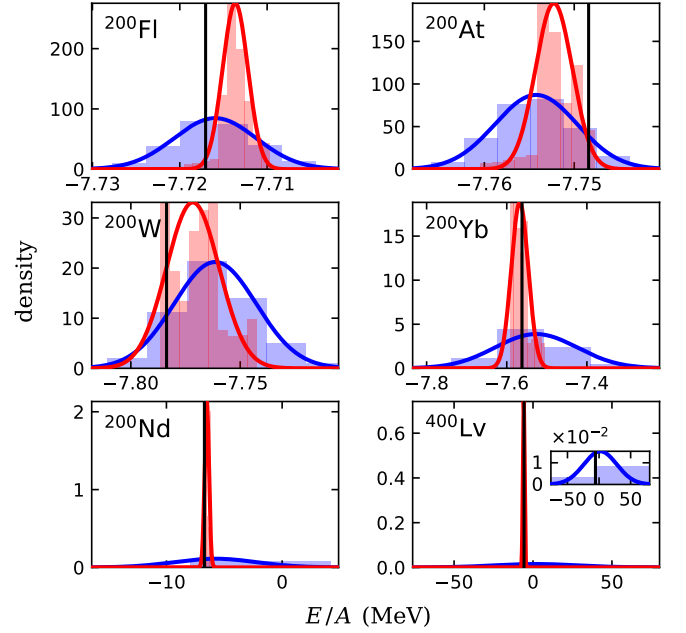


FIG. 10. Density distribution of the results from Δ -UQ (red) and the ensemble method (blue) for selected nuclei, for training on the DFT data. A Gaussian fit curve is shown along with each histogram. To clearly illustrate the fit, the figure is adjusted to the window of three standard deviations of the ensemble method results, with the vertical range adjusted to the maximum of both Δ -UQ and the ensemble Gaussian fit. The inset of the last figure is a zoom-in figure for the Gaussian fit of the ensemble results. The vertical black line is the true value.

IV. CONCLUSION

In this paper, we apply the Δ -UQ method to assess the epistemic uncertainty of machine learning models of the nuclear binding energy, which can be thought of as a simpler prototype of more complex deep-learning problems. We make use of two sets of data, one is calculated by density functional theory (DFT), the other is the AME2020 compilation. The AME2020 dataset is used to define the region (called “region I”) for training and validation for both AME2020 and DFT datasets. The DFT dataset has an additional testing set outside of the AME2020 region (called “region II”), which enables us to examine the reliability of the machine learning with Δ -UQ method when extrapolating far from stability.

We show that Δ -UQ gives accurate estimates of the uncertainty band, which follows the expected trend as its width widens when the evaluated nucleus is far from the training set nuclei. Furthermore, Δ -UQ ability to signal when machine learning results become unreliable as we extrapolate outward from the training region is superior to ensemble methods. The lack of fine-tuning control for the results produced by the thousands of individual ensemble runs can cause non-realistic, very large deviations in the extrapolation region. Therefore, ensemble methods can only estimate the upper bound of epistemic uncertainty. Instead, Δ -UQ estimates the epistemic uncertainty around the best fit, after all the machine learning

parameters have been carefully tuned. In addition, Δ -UQ is flexible in that it can be easily implemented to any deterministic or probabilistic neural network, by expanding the input dimension of the first linear layer. It can be used either for epistemic uncertainty quantification or simply to test the stability of training results responsive to changing initial weights. In contrast to the multiple runs needed by ensemble method to obtain epistemic uncertainties, Δ -UQ only needs one run, thereby significantly reducing the amount of computational resources.

ACKNOWLEDGMENTS

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Labora-

tory under Contract DE-AC52-07NA27344. This material is partially based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research and Office of Nuclear Physics, Scientific Discovery through Advanced Computing (SciDAC) program. Computing support for this work came from the Lawrence Livermore National Laboratory Institutional Computing Grand Challenge program. We are especially grateful to Jayaraman J. Thiagarajan and Vivek Narayanaswamy, co-inventors of the Δ -UQ method, for fruitful discussions.

-
- [1] R. Chandra and A. Rahmim, *Nuclear medicine physics: the basics* (Lippincott Williams & Wilkins, 2017).
 - [2] R. Devanathan, L. Van Brutzel, A. Chartier, C. Guéneau, A. E. Mattsson, V. Tikare, T. Bartel, T. Besmann, M. Stan, and P. Van Uffelen, Modeling and simulation of nuclear fuel materials, *Energy & Environmental Science* **3**, 1406 (2010).
 - [3] M. Wiescher, F. Käppler, and K. Langanke, Critical reactions in contemporary nuclear astrophysics, *Annual Review of Astronomy and Astrophysics* **50**, 165 (2012).
 - [4] J. A. Faber and F. A. Rasio, Binary neutron star mergers, *Living Reviews in Relativity* **15**, 8 (2012).
 - [5] F.-K. Thielemann, M. Eichler, I. Panov, and B. Wehmeyer, Neutron star mergers and nucleosynthesis of heavy elements, *Annual Review of Nuclear and Particle Science* **67**, 253 (2017).
 - [6] H. T. Janka, K. Langanke, A. Marek, G. Martínez-Pinedo, and B. Müller, Theory of core-collapse supernovae, *Physics Reports* **442**, 38 (2007).
 - [7] W. Huang, M. Wang, F. Kondev, G. Audi, and S. Naimi, The ame 2020 atomic mass evaluation (i). evaluation of input data, and adjustment procedures*, *Chinese Physics C* **45**, 030002 (2021).
 - [8] M. Wang, W. J. Huang, F. G. Kondev, G. Audi, and S. Naimi, The AME 2020 atomic mass evaluation (II). Tables, graphs and references, *Chin. Phys. C* **45**, 030003 (2021).
 - [9] L. Neufcourt, Y. Cao, S. A. Giuliani, W. Nazarewicz, E. Olsen, and O. B. Tarasov, Quantified limits of the nuclear landscape, *Phys. Rev. C* **101**, 044307 (2020).
 - [10] C. Gaulard, G. Audi, H. Doubre, S. Henry, D. Lunney, C. Monsanglant, M. de Saint Simon, C. Thibault, C. Toader, N. Vieira, G. Bollen, and C. Borcea, Exploring masses at the drip line: New mistral results, *AIP Conference Proceedings* **610**, 910 (2002).
 - [11] T. Glasmacher, B. Sherrill, W. Nazarewicz, A. Gade, P. Mantica, J. Wei, G. Bollen, and B. Bull, Facility for rare isotope beams update for nuclear physics news, *Nuclear Physics News* **27**, 28 (2017).
 - [12] Sherrill, Bradley M., Future opportunities at the facility for rare isotope beams, *EPJ Web Conf.* **178**, 01001 (2018).
 - [13] P. Möller, A. Sierk, T. Ichikawa, and H. Sagawa, Nuclear ground-state masses and deformations: Frdm(2012), *Atomic Data and Nuclear Data Tables* **109-110**, 1 (2016).
 - [14] W. Myers and W. Swiatecki, Nuclear properties according to the thomas-fermi model, *Nuclear Physics A* **601**, 141 (1996).
 - [15] G. Royer, M. Guibaud, and A. Onillon, Macro-microscopic mass formulae and nuclear mass predictions, *Nuclear Physics A* **847**, 24 (2010).
 - [16] J. Duflo, Phenomenological calculation for nuclear masses and charge radii, *Nuclear Physics A* **576**, 29 (1994).
 - [17] A. P. Zuker, On the microscopic derivation of a mass formula, *Nuclear Physics A* **576**, 65 (1994).
 - [18] W. Ryssens, G. Scamps, S. Goriely, and M. Bender, Skyrme–Hartree–Fock–Bogoliubov mass models on a 3D mesh: II. Time-reversal symmetry breaking, *Eur. Phys. J. A* **58**, 246 (2022).
 - [19] S. Goriely, N. Chamel, and J. M. Pearson, Hartree-Fock–Bogoliubov nuclear mass model with 0.50 MeV accuracy based on standard forms of Skyrme and pairing functionals, *Phys. Rev. C* **88**, 061302 (2013).
 - [20] S. Goriely, S. Hilaire, M. Girod, and S. Péru, First Gogny–Hartree–Fock–Bogoliubov Nuclear Mass Model, *Phys. Rev. Lett.* **102**, 242501 (2009).
 - [21] E. Yüksel, D. Soydaner, and H. Bahtiyar, Nuclear mass predictions using machine learning models, *Phys. Rev. C* **109**, 064322 (2024).
 - [22] A. E. Lovell, A. T. Mohan, T. M. Sprouse, and M. R. Mumpower, Nuclear masses learned from a probabilistic neural network, *Phys. Rev. C* **106**, 014305 (2022).
 - [23] Z.-P. Gao, Y.-J. Wang, H.-L. Lü, Q.-F. Li, C.-W. Shen, and L. Liu, Machine learning the nuclear mass, *Nuclear Science and Techniques* **32**, 109 (2021).
 - [24] M. Li, T. M. Sprouse, B. S. Meyer, and M. R. Mumpower, Atomic masses with machine learning for the astrophysical r process, *Physics Letters B* **848**, 138385 (2024).
 - [25] M. R. Mumpower, T. M. Sprouse, A. E. Lovell, and A. T. Mohan, Physically interpretable machine learning for nuclear masses, *Phys. Rev. C* **106**, L021301 (2022).
 - [26] T. Bayram, C. M. Yeşilkanat, and S. Akkoyun, Applications of different machine learning methods on nuclear charge radius estimations, *Physica Scripta* **98**, 125310 (2023).
 - [27] X.-X. Dong, R. An, J.-X. Lu, and L.-S. Geng, Nuclear charge radii in bayesian neural networks revisited, *Physics Letters B* **838**, 137726 (2023).
 - [28] C. Jin, T. Li, J. Zhang, W. Zhang, B. Yang, R. Ren, and C. Cui, Fecsg-ml: Feature engineering for nuclear reaction cross sec-

- tions generation using machine learning, *Applied Radiation and Isotopes* **214**, 111545 (2024).
- [29] A. Bari, T. P. Garg, Y. Wu, S. Singh, and D. Nagel, Exploring artificial intelligence techniques to research low energy nuclear reactions, *Front Artif Intell* **7**, 1401782 (2024), 2624-8212 Bari, Anasse Garg, Tanya Pushkin Wu, Yvonne Singh, Sneha Nagel, David Journal Article Review Switzerland 2024/09/09 Front Artif Intell. 2024 Aug 23;7:1401782. doi: 10.3389/frai.2024.1401782. eCollection 2024.
- [30] L. Neufcourt, Y. Cao, W. Nazarewicz, and F. Viens, Bayesian approach to model-based extrapolation of nuclear observables, *Phys. Rev. C* **98**, 034318 (2018).
- [31] L. Neufcourt, Y. Cao, W. Nazarewicz, E. Olsen, and F. Viens, Neutron Drip Line in the Ca Region from Bayesian Model Averaging, *Phys. Rev. Lett.* **122**, 062502 (2019).
- [32] B. Lakshminarayanan, A. Pritzel, and C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, in *Advances in Neural Information Processing Systems*, Vol. 30, edited by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Curran Associates, Inc., 2017).
- [33] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, and J. Snoek, Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift, in *Advances in Neural Information Processing Systems*, Vol. 32, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Curran Associates, Inc., 2019).
- [34] R. Rahaman and A. H. Thiery, Uncertainty quantification and deep ensembles, in *Advances in Neural Information Processing Systems*, Vol. 34, edited by M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan (Curran Associates, Inc., 2021) pp. 20063–20075.
- [35] M. Valdenegro-Toro, Sub-ensembles for fast uncertainty estimation in neural networks, in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops* (2023) pp. 4119–4127.
- [36] A. R. Tan, S. Urata, S. Goldman, J. C. B. Dietschreit, and R. Gómez-Bombarelli, Single-model uncertainty quantification in neural network potentials does not consistently outperform model ensembles, *npj Computational Materials* **9**, 225 (2023).
- [37] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, A survey of uncertainty in deep neural networks, *Artificial Intelligence Review* **56**, 1513 (2023).
- [38] J. J. Thiagarajan, R. Anirudh, V. Narayanaswamy, and P. Bremer, Single model uncertainty estimation via stochastic data centering, in *Advances in Neural Information Processing Systems*, Vol. 35, edited by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Curran Associates, Inc., 2022) pp. 8662–8674.
- [39] J. J. Thiagarajan, V. Narayanaswamy, P. Trivedi, and R. Anirudh, PAGER: Accurate failure characterization in deep regression models, in *Proceedings of the 41st International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 235, edited by R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp (PMLR, 2024) pp. 21069–21082.
- [40] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization* (2017), [arXiv:1412.6980 \[cs.LG\]](https://arxiv.org/abs/1412.6980).
- [41] E. Chabanat, P. Bonche, P. Haensel, J. Meyer, and R. Schaeffer, A Skyrme parametrization from subnuclear to neutron star densities, *Nucl. Phys. A* **627**, 710 (1997).
- [42] T. Li, N. Schunck, and M. Grosskopf, Multipole responses in fissioning nuclei and their uncertainties, *Phys. Rev. C* **110**, 034317 (2024).
- [43] R. Navarro Pérez and N. Schunck, Controlling extrapolations of nuclear properties with feature selection, *Phys. Lett. B* **833**, 137336 (2022).
- [44] S. Perez-Martin and L. Robledo, Microscopic justification of the equal filling approximation, *Phys. Rev. C* **78**, 014304 (2008).
- [45] P. Marević, N. Schunck, E. M. Ney, R. Navarro Pérez, M. Verriere, and J. O'Neal, Axially-deformed solution of the Skyrme-Hartree-Fock-Bogoliubov equations using the transformed harmonic oscillator basis (IV) HFBTHO (v4.0): A new version of the program, *Comput. Phys. Commun.* **276**, 108367 (2022).
- [46] S. Ioffe and C. Szegedy, *Batch normalization: Accelerating deep network training by reducing internal covariate shift* (2015), [arXiv:1502.03167 \[cs.LG\]](https://arxiv.org/abs/1502.03167).
- [47] D. C. Hoaglin, B. Iglewicz, and J. W. Tukey, Performance of some resistant rules for outlier labeling, *Journal of the American Statistical Association* **81**, 991 (1986).
- [48] K. He, X. Zhang, S. Ren, and J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (2015) pp. 1026–1034.